NVIDIA NETWORKING • DEEP DIVE

ConnectX-6 RoCEv2

拥塞算法与调优专题

DCQCN / ECN / PFC 的算法原理、参数敏感性、实验设计与分场景调优

算法原理 · 参数敏感性 · 实验设计 · 分场景调优 · 21 份依据

与主指南的关系

配套《ConnectX-6 网卡 RoCEv2 拥塞控制与 PFC 流控实施指南 v1.1》。

主指南回答“怎么配”，本专题回答“为什么这样调、如何证明调对了”。

适用对象

网络架构师、RDMA 性能工程师、AI/HPC 集群调优与测试人员

版本	日期	文档状态
v1.0	**2026-10-25**	**技术专题稿 · 与主指南配套**

本文件为公开资料的工程整理；算法结论以 NVIDIA 文档与原始论文为准，调优建议需现场验证。

文档信息

项目	内容
文档名称	ConnectX-6 RoCEv2 拥塞算法与调优专题
副标题	DCQCN / ECN / PFC 的算法原理、参数敏感性、实验设计与分场景调优
版本 / 日期	v1.0 / 2026-10-25
证据基线	21 份来源：NVIDIA 官方文档、DOCA/MFT 文档、RFC、ACM 论文
适用平台	ConnectX-6 EN/VPI 的 Ethernet 模式；RoCEv2；Linux
配套文档	《ConnectX-6 网卡 RoCEv2 拥塞控制与 PFC 流控实施指南 v1.1》

与主指南的分工

文档	重点	回答的问题
主指南	机制定位、端到端设计、命令、验收、排障、回滚	回答“怎么配、怎么验”
本专题	算法原理、参数敏感性、实验设计、分场景调优配方	回答“为什么这样调、如何调得更稳”
共享结论	PFC 是逐跳流控；DCQCN 是端到端 CC；CX6 主线是 ECN+CNP+DCQCN	两份文档保持一致

阅读说明与证据等级

正文用 [C1]–[C21] 标注来源。NVIDIA 文档用于能力、默认值、命令和平台边界；RFC 用于协议语义；ACM 论文用于算法来源与设计背景。工程建议和工程公式会单独标注，不与官方事实混淆。

等级	含义	标记
官方事实	NVIDIA 产品文档中的能力、默认值、命令、限制	以 [C#] 标注
标准/论文	RFC 的协议语义、ACM 论文的算法设计	以 [C#] 标注
工程建议	调优顺序、实验方法、经验阈值	文中写“建议/通常/起始假设”
工程公式	BDP、队列时延、headroom、Jain 指数等	文中标注“工程公式”

目录

章	内容	章	内容
1	专题导读：从“能跑”到“跑稳”	7	参数敏感性与交互
2	拥塞控制基础：指标、时间尺度与闭环	8	调优方法论：基线、实验与统计
3	算法地图：DCQCN、QCN、ZTR-RTT、PCC	9	分场景调优配方
4	ECN / RED / WRED：交换机侧信号	10	调优案例与决策树
5	DCQCN 深度讲解	11	结论与检查清单
6	其他算法：QCN、ZTR-RTT、PCC	A	附录 A 公式与量纲
B	附录 B 实验记录模板	C	附录 C 证据摘录
D	附录 D 来源与链接		

1 专题导读：从“能跑”到“跑稳”

“能跑”只需要链路通、RoCE 能建链；“跑稳”要求在 incast、突发、故障、混合流量下，吞吐、时延、公平性和丢包都可控。两者之间差的不是某一个参数，而是一套可复现的调优方法。

1.1 为什么默认值不够

- NVIDIA 默认值是通用起点，不是为你的 incast 比例、MTU、RTT、业务突发模式选的。[C4][C5]
- 同一个 ECN 阈值，在 100G/100m 和 200G/2km 上对应的队列时延完全不同；只抄数字没有意义。
- PFC 和 ECN 作用在不同缓冲：PFC 看入口 PG，ECN 看出口队列；不能用一个阈值去“对齐”另一个。
- 业务目标不同：AI 训练怕长尾和同步突发，存储怕丢包和重传，多租户怕相互影响；调优方向不可能一样。
- 默认值 + 错误优先级（比如 CNP 没走 strict）时，调任何 CC 参数都可能无效。

1.2 本专题要建立的三个能力

能力	目标	章节
看懂算法	知道 DCQCN 为什么用 ECN/CNP、α 和 RPG_GD 在做什么、ZTR-RTT/PCC 为什么不是 CX6 的默认路径	第 2–6 章
看懂参数	知道每个参数调大/调小会改变什么、和谁耦合、先调谁	第 7 章
看懂实验	能用单流、incast、突发、混合流量证明“调对了”，而不是只看峰值吞吐	第 8–10 章

1.3 三条调优原则

1. **先正确，再优化**：优先级、CNP QoS、PFC 域、buffer/headroom 不正确时，不要调 DCQCN 参数。
2. **一次只改一个变量**：同时改 Kmin、RPG_GD、AI/HAI，出了问题无法归因。
3. **用分布，不用平均值**：看 P50/P99/P999、ECN 标记率、PFC pause、Jain 公平性，不只看平均吞吐。

> **调优不是“找一组神参数”**
> DCQCN 的稳定性取决于交换机标记、CNP 回送、端点速率控制和业务负载的共同作用。把某组参数从别人的集群抄过来，往往比默认值更差。

1.4 一页结论

主题	结论	依据
算法	CX6 的 RoCEv2 主线是 DCQCN；QCN 是 L2 遗留；ZTR-RTT/PCC 属于更新平台/框架。	[C1][C4][C7][C9]
信号	ECN 是早期信号，CNP 是反馈，PFC 是逐跳兜底；三者不能互相替代。	[C3][C4][C11]
调优顺序	正确性 → 缓冲/headroom → ECN 阈值 → CNP QoS → DCQCN 恢复参数 → PFC 微调。	工程建议

主题	结论	依据
验证	单流、4:1/8:1 incast、突发、混合流量、重启持久化，至少五类。	工程建议
安全	每次变更都要有基线、回滚、停止条件；不要在不知道原值的情况下改参数。	工程建议

2 拥塞控制基础：指标、时间尺度与闭环

拥塞控制不是“把队列压到 0”。它是在吞吐、时延、公平性、丢包之间做权衡。理解这个权衡，才能判断一个参数该不该动。

2.1 拥塞从哪里来

来源	典型场景	表现	优先动作
持续超配	下行带宽小于上行汇聚带宽	队列持续增长，ECN/PFC 长期生效	扩容/限速/流量工程
Incast	多个发送端同时打一个接收端/一个出口	微秒级队列尖峰，平均利用率可能不高	ECN 早期标记、PFC 兜底、应用限流
Microburst	短时间线速突发，平均速率不高	PFC 触发但 ECN 不一定标记	headroom、入口缓冲、 pacing
速度不匹配	100G 进 25G 出、或交换机内部/外部速率差	出口队列堆积	QoS/整形/端口规划
慢接收端	应用不 drain、QP 饥饿、CPU/GPU 瓶颈	PFC 反压扩散，甚至风暴	修应用/接收端，不是调 CC
多租户/混合流量	TCP 与 RoCE 共享 buffer/priority	互相挤占，PFC 误伤	优先级隔离、独立 lossless pool

2.2 控制回路：谁在控制谁

把拥塞控制看成闭环：交换机队列是被控对象，ECN 是传感器，CNP 是反馈通道，发送端限速器是执行器。调参本质上是在改控制器增益和反馈延迟。

角色	位置	观测量	可调项
被控对象	交换机 Egress 队列 / 入口 PG	队列长度、占用率、丢弃	buffer、阈值、调度
传感器	ECN CE 标记	标记概率、标记率	Kmin/Kmax/Pmax
反馈	CNP	CNP 速率、时延、丢失	CNP DSCP/PCP、strict、最小间隔
执行器	发送端 RP 限速器	CurrentRate/TargetRate	α、RPG_GD、AI/HAI、定时器
兜底	PFC	Pause 帧、pause duration	Xoff/Xon、headroom、watchdog

2.3 时间尺度

阶段	时间尺度	发生什么	调优含义
PFC 反应	微秒级	入口缓冲到 Xoff，发 Pause；链路两端 MAC/交换芯片处理	PFC 是最后一道防线
ECN 标记	微秒–毫秒	Egress 队列超过 Kmin，按概率标记	标记越早，越早给端点信号
CNP 反馈	RTT 量级	接收端生成 CNP，回到发送端	CNP 延迟/丢失会直接拖慢反应

阶段	时间尺度	发生什么	调优含义
α 更新	DCE_TCP_RTT（默认 1 µs）	有/无 CNP 的 EWMA	默认极快，不要按“RTT”字面理解
降速窗口	RATE_REDUCE_MONITOR_PERIOD（默认 4 µs）	观察 CNP 后降速	窗口越短越敏感
恢复	RPG_TIME_RESET 300 µs / BYTE_RESET ~2 MB	AI/HAI 逐步升速	恢复过快会再次拥塞

> **DCE_TCP_RTT 不是实测 RTT**
> DOCA 文档给的单位是 µs、默认 1；它是 α 更新周期，不是路径 RTT 测量。把它当成“RTT”会误判调优方向。[C5]

2.4 指标：不要只看平均吞吐

指标	定义/来源	维度	注意
吞吐/Goodput	完成的有用数据速率	平均 + 分位	平均值掩盖长尾
时延	P50/P99/P999、队列时延	分位 + 时间序列	P99 才能看到 incast/排队
丢包	`no-buffer-discard`、重传、out-of-sequence	计数 + 速率	lossless 下应为 0 或接近 0
ECN	标记率、标记包数	按 TC/端口	标记率高不一定是坏事，要看队列
PFC	pause 包数/时长、storm 计数	按 priority	高 pause + 低 ECN = PFC 在替 ECN 干活
公平性	Jain 指数、各流吞吐分布	多流/多租户	平均值相同也可能极不公平
收敛/稳定	恢复时间、振荡幅度	阶跃/incast 后	只看稳态会漏掉振荡

Jain 公平性指数（工程公式）：`J = (Σx_i)^2 / (n · Σx_i^2)`。J=1 表示完全公平；J 越低越不公平。它不替代业务 SLO，但能揭示“总吞吐正常、个别流饿死”的问题。

2.5 为什么 RoCE 用 ECN 而不是丢包

- RoCE 的丢包恢复依赖重传/乱序处理，代价比 TCP 更高；lossless 设计希望“提前降速”而不是“丢了再补”。[C4]
- ECN 在 IP 头标记 CE，不丢包；接收端网卡生成 CNP，发送端按 QP 降速，形成闭环。[C4][C19]
- PFC 是逐跳暂停，不告诉发送端“降多少”；它只能兜底，不能替代 CC。[C3][C11]
- 因此 RoCEv2 的首选信号是 ECN，PFC 是最后一道防线；两者职责不同。

3 算法地图：DCQCN、QCN、ZTR-RTT、PCC

先把“算法”与“机制”分开：PFC 是流控，QCN/DCQCN/ZTR-RTT 是拥塞控制，PCC 是可编程框架。把它们混在一起，就会得出“开 PFC 就能替代 CC”这类错误结论。

3.1 一张图

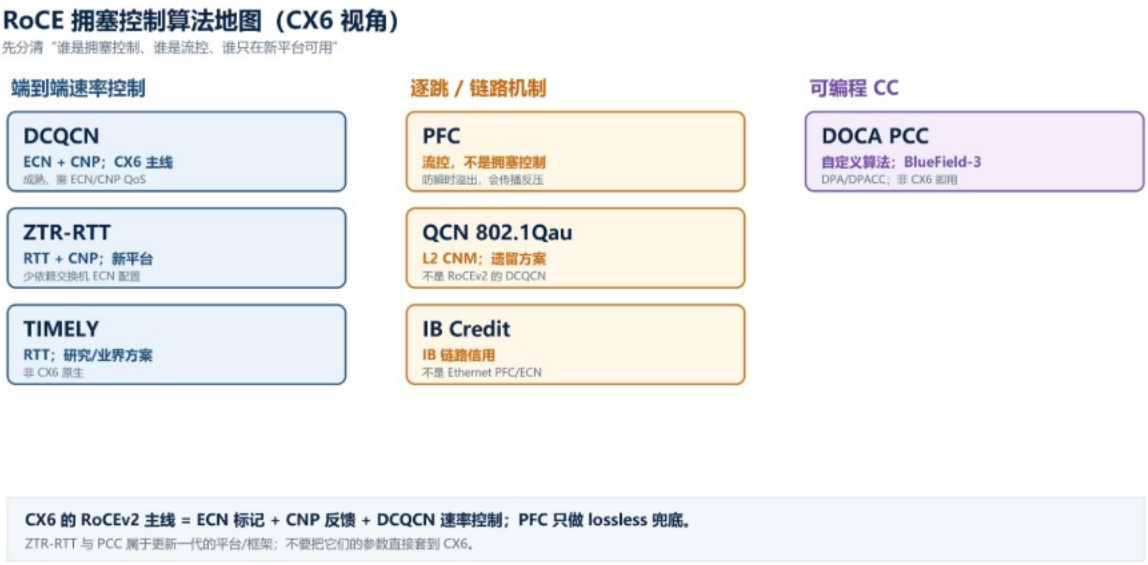


图 1 RoCE 拥塞控制算法地图（按 [C1][C4][C7][C9][C11] 整理，概念图）

3.2 算法家族

家族	代表	核心信号/动作	适用与边界	依据
丢包型	TCP Reno 等	以丢包作为拥塞信号，AIMD/慢启动	通用互联网/TCP；RoCE 不希望丢包，恢复代价高	[C21]
ECN 型	DCTCP、DCQCN	ECN 标记 + 端点降速；DCQCN 用 CNP 反馈	RoCEv2/数据中心；CX6 主线，需交换机支持 ECN	[C4][C5]
延迟型	TIMELY、ZTR-RTT	RTT/排队时延作为信号；AIMD/窗口	数据中心/新平台；ZTR 在新平台，TIMELY 为研究方案	[C7][C20]
L2 CNM	QCN（802.1Qau）	交换机生成 CNM，端点降速	Legacy L2/RoCEv1；L2、不可路由	[C1]
信用	IB Credit	链路信用，逐跳保证不溢出	InfiniBand；不是 Ethernet PFC/ECN	[C1]
可编程	DOCA PCC	用户实现 CC 算法，DPA 执行	BlueField-3/新平台；开发门槛高，非 CX6 即用	[C9]

3.3 CX6 上应该用哪一个

算法	定位	在 CX6 上的用法	依据
DCQCN	CX6 的 RoCEv2 主线；ECN + CNP + 速率控制	默认选它；先保证 ECN/CNP QoS 正确	[C1][C4][C5]

算法	定位	在 CX6 上的用法	依据
QCN	CX6 手册列有 802.1Qau 能力；L2 遗留	只有既有 L2/QCN 环境才考虑；不要与 DCQCN 混用	[C1]
ZTR-RTT	DOCA 3.5 的新一代原生算法；RTT+CNP	不要假设 CX6 支持；按型号/固件文档确认	[C6][C7]
PCC	可编程 CC；BlueField-3 + DPA	CX6 不按 PCC 方式配置；需要新平台/开发投入	[C9]
TIMELY	RTT 型研究方案	用于理解延迟型 CC，不作为 CX6 配置项	[C20]

3.4 选择算法时的四个问题

1. 业务能否容忍丢包/重传？不能 → lossless + ECN/PFC；能 → lossy + ECN/重传。
2. 路径是否可路由/跨 L3？是 → ECN/DCQCN 或 ZTR-RTT；否 → 可考虑 L2/QCN，但优先 DCQCN。
3. 交换机是否支持 ECN/CNP QoS？否 → 先补网络能力，或评估 ZTR-RTT 平台；不要把 DCQCN 硬开在无标记路径上。
4. 是否愿意/能够开发自定义算法？是 → PCC（新平台）；否 → 用原生 DCQCN/ZTR-RTT。

> **不要把“算法”当“配置开关”**
> DCQCN 不是网卡上的一个 enable 位；它依赖交换机 ECN 标记、CNP 的 QoS、端点参数和缓冲设计。只开 `ROCE_CC_PRIO_MASK` 而不配 ECN/CNP，等于闭环断了两段。

4 ECN / RED / WRED：交换机侧信号

ECN 是 DCQCN 的“传感器”。交换机只负责在队列增长时把 ECT 包标记为 CE；它不直接降速，降速由端点完成。把 ECN 配成“要么不标记、要么全标记”，闭环都会失效。

4.1 ECN 的协议语义

ECN 用 IP 头 2 bit 表示：00 Not-ECT、01/10 ECT、11 CE。支持 ECN 的交换机在队列超过阈值时把 ECT 改成 CE；接收端看到 CE 后生成 CNP。它要求路径上所有节点支持 ECN，否则标记无法端到端传递。[C2][C19]

ECN 位	名称	含义
00	Not-ECT	不支持 ECN；拥塞时通常丢弃
01	ECT(1)	支持 ECN，可被标记为 CE
10	ECT(0)	支持 ECN，可被标记为 CE
11	CE	已遇到拥塞；接收端生成 CNP

4.2 标记曲线

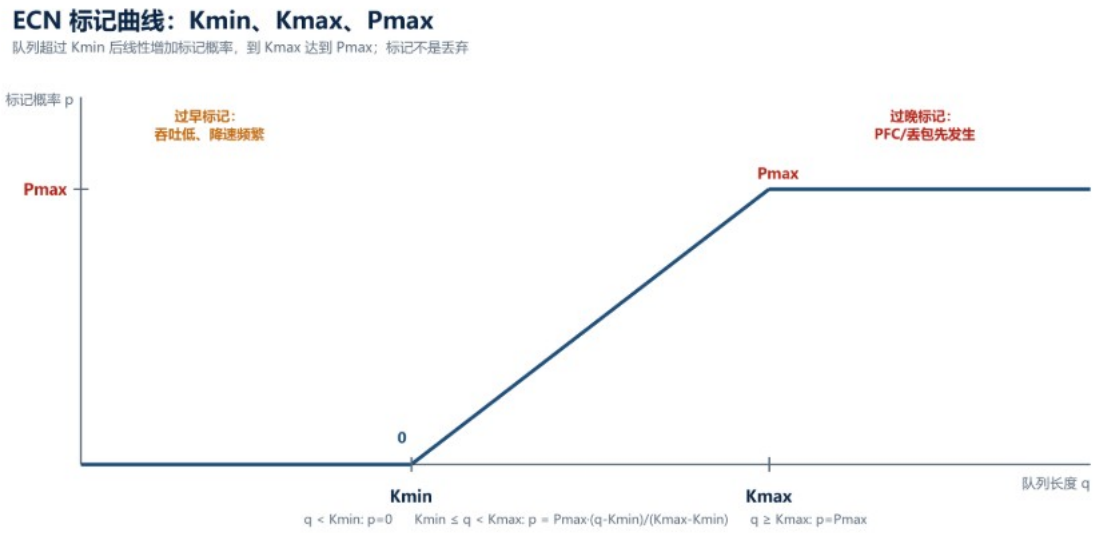


图 2 ECN 标记概率与 Kmin/Kmax/Pmax 的关系（概念图，依据 [C4][C13]）

DCQCN 的拥塞点通常是交换机 Egress 队列：q < Kmin 不标记；Kmin ≤ q < Kmax 线性增加；q ≥ Kmax 达到 Pmax。Cumulus 的默认 RoCE profile 把 Pmax 设为 100%，即在 Kmax 时所有 ECN-capable 包都被标记。[C4][C14]

```
q < Kmin:                 p = 0
Kmin <= q < Kmax:         p = Pmax * (q - Kmin) / (Kmax - Kmin)
q >= Kmax:                p = Pmax

工程换算：
队列时延 qdelay ≈ q / C
Kmin 对应的时延 ≈ Kmin / C
```

4.3 RED、WRED 与 ECN 的区别

机制	全称	动作	典型用途	信号
RED	随机早期检测	队列超过阈值时按概率丢弃	有损流量/TCP	用丢包通知拥塞
WRED	加权 RED	按颜色/优先级使用不同阈值	DiffServ/多优先级	更细的丢弃策略
ECN	显式拥塞通知	把 ECT 标记为 CE，不丢弃	RoCEv2/DCQCN	用标记通知拥塞

对 lossless RoCE，优先 ECN；RED 可作为有损流量的补充，但不要让 RED 先于 ECN 丢掉 RoCE 包。Onyx 支持 `traffic-class <tc> congestion-control [red|ecn|both]`，阈值可用绝对 KB 或相对百分比；两者不要混着配。[C13]

4.4 Onyx 与 Cumulus 的配置差异

平台	配置入口	单位/注意	依据
Onyx	`traffic-class <tc> congestion-control ecn minimum-absolute <KB> maximum-absolute <KB>`	绝对阈值单位 KB；也支持 relative 百分比	[C13][C10]
Cumulus	`nv set qos congestion-control default-global traffic-class <tc> min-threshold <bytes> max-threshold <bytes>`	默认 profile 以字节表示；RoCE profile 用 TC3	[C14][C15]
共同点	Kmin/Kmax/Pmax 语义一致；都要按 TC、按端口核对	不要把 Onyx 的 150 当字节，也不要把 Cumulus 的 150000 当 KB	[C13][C14]

4.5 从“队列时延”理解阈值

把 Kmin 换算成时间，比记数字更有意义：`Kmin_time ≈ Kmin_bytes / C`。同样 150 KB，在 100G 上约 12 µs，在 200G 上约 6 µs；同样的阈值在不同速率下代表不同的排队时延。

线速	字节速率	100 µs RTT 的 BDP	150 KB 对应队列时延
100G	12.5 GB/s	1.25 MB	150 KB ≈ 12 µs
200G	25 GB/s	2.5 MB	150 KB ≈ 6 µs
400G	50 GB/s	5 MB	150 KB ≈ 3 µs

工程含义：不要问“Kmin 应该是多少字节”，先问“我希望 ECN 在多少 µs 队列时延开始标记”。目标时延由业务 SLO、RTT、交换机缓冲和 PFC 阈值共同决定。

4.6 常见错误

- 把 Onyx 的 150 当字节，导致 ECN 几乎不标记；或把 Cumulus 的 150000 当 KB，导致过早全标记。
- 只开 ECN、不开 CNP 的 strict QoS；标记了但反馈回不来。
- Kmin 设得比 PFC Xoff 还晚，PFC 先触发，DCQCN 看不到持续拥塞。
- Pmax 设得很低，队列已经很高却只标记少量包，端点降速不足。
- 用 RED 替 ECN 丢 RoCE 包，再指望 DCQCN 恢复；方向反了。

5 DCQCN 深度讲解

DCQCN 是 CX6 上 RoCEv2 的主线算法。它把 DCTCP 的 ECN 拥塞评分和 QCN 的速率控制引擎结合起来：用 ECN 感知拥塞，用 CNP 回送反馈，用 α 和乘法降速/加法恢复调节每个 QP 的速率。[C4][C18]

5.1 三角色与闭环

1. **CP（拥塞点）**：交换机 Egress 队列按 Kmin/Kmax/Pmax 标记 CE。[C4]
2. **NP（通知点）**：接收端网卡看到 CE，生成 CNP，按 QP 回送；硬件完成，不经过 CPU。[C4]
3. **RP（反应点）**：发送端网卡收到 CNP，更新 α，降速；没有 CNP 时逐步恢复。[C4]
4. **闭环**：降速 → 队列下降 → 停止标记 → CNP 减少 → 升速；再次拥塞则重复。

CNP 是按流/QP 反馈的：一个 QP 拥塞，不会让同网卡所有 QP 一起降速。这是 DCQCN 相对全局 pause 的关键优势，也是它必须和 PFC 分开理解的原因。[C4]

5.2 状态机

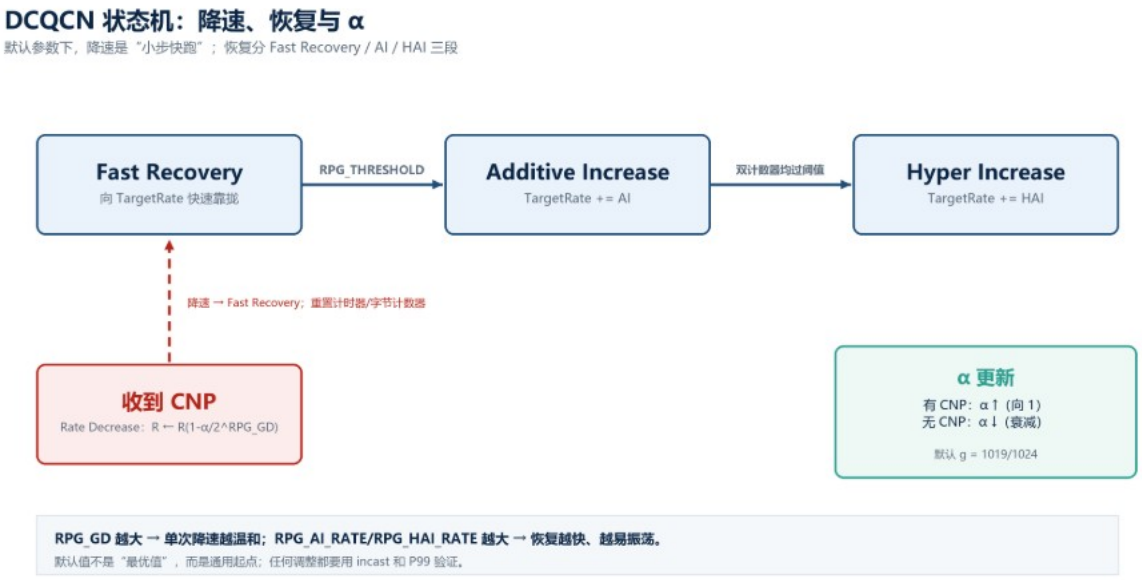


图 3 DCQCN 的降速、Fast Recovery、AI、HAI 与 α 更新（依据 [C4][C5] 绘制）

收到 CNP 后先做乘法降速，再进入 Fast Recovery；连续无 CNP 的触发达到阈值后进入 AI，再达到双计数器阈值后进入 HAI。α 表示近期拥塞程度，随 CNP 出现而升高、随静默而衰减。[C4]

5.3 公式

```
alpha 更新（每个 DCE_TCP_RTT）：
  有 CNP:   alpha <- (1 - g) * alpha + g
  无 CNP:   alpha <- (1 - g) * alpha
  g = DCE_TCP_G / 1024

降速（收到 CNP，按 RATE_REDUCE_MONITOR_PERIOD 观察）：
  CurrentRate <- CurrentRate * (1 - alpha / 2^RPG_GD)

恢复（无 CNP）：
  CurrentRate <- (CurrentRate + TargetRate) / 2
  TargetRate  <- TargetRate + AI    (Additive Increase)
  TargetRate  <- TargetRate + HAI   (Hyper Increase)
```

默认 `DCE_TCP_G=1019`、`RPG_GD=11`。按公式直算，α=1 时单次降速约 1/2048；所以 DCQCN 的默认行为是“小步快跑”，依赖持续的 CNP 和 4 µs 的监控窗口，而不是一次 CNP 砍半。把 `RPG_GD` 调小会让单次降速更狠，调大更温和。[C5]

5.4 参数与行为

参数	默认	作用	调优优先级
ROCE_CC_PRIO_MASK	Enable（逐优先级）	按 priority 启用 DCQCN；255=全开	默认优先 [C5][C17]
DCE_TCP_G	1019	α 的 EWMA 权重	默认优先 [C5]
DCE_TCP_RTT	1 µs	α 更新周期	默认优先 [C5]
INITIAL_ALPHA_VALUE	1023	首个 CNP 的 α	默认优先 [C5]
RATE_TO_SET_ON_FIRST_CNP	0	首个 CNP 是否强制速率	默认优先 [C5]
RPG_GD	11	降速指数；越小越狠	谨慎 [C5]
RPG_MIN_DEC_FAC	50%	单次降速上限	谨慎 [C5]
RPG_MIN_RATE	1 Mb/s	QP 最低速率	谨慎 [C5]
RATE_REDUCE_MONITOR_PERIOD	4 µs	CNP 观察窗口	谨慎 [C5]
RPG_TIME_RESET	300 µs	升速定时器	谨慎 [C5]
RPG_BYTE_RESET	32767×64 B≈2 MB	升速字节计数器	谨慎 [C5]
RPG_THRESHOLD	1	AI/HAI 的连续触发次数	谨慎 [C5]
RPG_AI_RATE / HAI_RATE	5 / 50 Mb/s	AI/HAI 步长	谨慎 [C5]
CNP_DSCP / CNP_802P_PRIO	48 / 6	CNP 的 DSCP/PCP	先保证 QoS [C5]
MIN_TIME_BETWEEN_CNPS	4 µs	CNP 最小间隔	谨慎 [C5]

5.5 DCQCN 的公平性与收敛

- **按 QP 反馈**：每个 QP 独立降速；天然支持多流，但也意味着 QP 数越多，控制面越复杂。
- **RTT 偏置**：长 RTT 流收到 CNP 更晚、恢复更慢，可能出现不公平；DCQCN 本身不保证 RTT 公平。
- **α 的记忆**：默认 g 很大，α 对单个 CNP 很敏感、对静默衰减也快；这使系统反应快，但也更容易受 CNP 到达模式影响。
- **收敛不是无条件**：交换机阈值、CNP 速率、QP 数、RTT、PFC 交互都会影响收敛；必须用 incast 实测，不能只看公式。

- **PFC 会掩盖拥塞**：如果 PFC 先触发，发送端可能收不到足够 CNP，速率不降，PFC 变成主要反压；这是“PFC 很忙、ECN 很闲”的根因之一。

5.6 DCQCN vs DCTCP vs QCN

算法	层/信号	反馈	执行	定位
DCTCP	TCP/ECN	ACK 携带 ECN-Echo	软件 TCP	数据中心 TCP
DCQCN	RoCEv2/ECN	CNP 硬件回送	网卡硬件	CX6 RoCE 主线
QCN	L2/802.1Qau	CNM	网卡/交换机	Legacy L2

DCQCN 不是“更高级的 QCN”，而是为可路由 RoCEv2 重新设计的 ECN/CNP 闭环；QCN 的 L2 CNM 不能跨路由域，不能拿来替代 DCQCN。[C1][C4]

5.7 DCQCN 做不到什么

- 不能创造带宽；持续超配/严重 incast 需要容量/流量工程。
- 不能替代 PFC；严重瞬时溢出仍需入口缓冲/PFC 兜底。
- 不能感知不标记 ECN 的瓶颈；路径上有一个“盲区”，闭环就缺一段。
- 不能修复 CNP 丢失/被 pause/映射错误；反馈断了，RP 会误判。
- 不能保证 RTT 公平；长 RTT 流可能吃亏。

6 其他算法：QCN、ZTR-RTT、PCC

这一章把“与 CX6 相关但非主线”的算法讲清楚，目的不是让你都去用，而是避免把不同代际、不同平台的算法混在一起调。

6.1 QCN（802.1Qau）：L2 的遗留方案

QCN 是 IEEE 802.1Qau 定义的 Layer 2 拥塞通知：交换机检测到拥塞后生成 CNM（Congestion Notification Message），端点收到后降速。ConnectX-6 手册列有 802.1Qau 能力，但这不等于 RoCEv2 的 DCQCN。[C1][C4]

维度	QCN	DCQCN
层	Layer 2 / 不可路由	DCQCN 在 IP/ECN 上，可跨 L3
反馈	CNM	CNP
主要场景	Legacy L2 / RoCEv1	RoCEv2 / 现代数据中心
与 CX6 关系	硬件能力存在	RoCEv2 主线是 DCQCN

> **不要把 QCN 当 DCQCN 调**
> QCN 的 CNM 和 DCQCN 的 CNP 不是同一个反馈；QCN 的参数、计数器、交换机要求也不同。只有在既有 L2/QCN 环境中才考虑 QCN，RoCEv2 优先 DCQCN。

6.2 ZTR-RTT：延迟型的新一代原生算法

NVIDIA DOCA 3.5 把 ZTR-RTT 描述为 Zero Touch RoCE 的硬件拥塞控制算法：它同时使用 RTT 和 CNP，比较实测 RTT 与目标 RTT，按 AIMD 调整 cwnd/速率；通过硬件时间戳测量 RTT，减少对交换机 ECN 阈值配置的依赖。[C7]

- **信号**：RTT + CNP；RTT 由发送端/接收端硬件时间戳协作测量。[C7]
- **动作**：Additive Increase / Multiplicative Decrease；以 cwnd（字节）收敛到速率。[C7][C8]
- **特点**：首次拥塞快速反应；可减少 HOL 阻塞；目标是“zero touch”部署。[C7]
- **平台**：DOCA 将其放在 DPA-based CC engine/新一代原生 CC 基础设施中；不要假设 CX6 支持，按型号/固件文档确认。[C6][C7]

参数	含义	类型	默认/备注
ALPHA	目标 RTT 与 base RTT 的线性系数	fxp16	0.1
MAX_DEC	单次降速因子	fxp16	0.97
MAX_INC	单次升速因子	fxp16	1.06
AI	每 100Gb/s 的加性增量（字节）	integer	按平台

这些参数只用于说明 ZTR-RTT 的调节维度，**不是 CX6 DCQCN 的参数**。把 ALPHA/MAX_DEC/MAX_INC 套到 CX6 上没有意义。[C8]

6.3 DOCA PCC：把算法变成可编程

DOCA PCC 提供 RP/NP 的可编程接口：用户用 DPACC 编译自定义 CC 逻辑，在 BlueField-3 的 DPA 上执行；可配置 probe 包、CC 事件、每流速率限制等。[C9]

- **平台**：DOCA PCC 文档要求 BlueField-3（或更新）平台；不是 CX6 的即用功能。[C9]
- **启用**：RP 侧 `USER_PROGRAMMABLE_CC=1`，NP 侧 `PCC_INT_EN=0`；需要关机/重启。[C9]
- **开发**：需要 DPACC 编译、host library、device library；不是改几个 sysfs 就能完成。[C9]
- **适用**：研究/差异化算法、需要自定义信号（CCMAD/IFA）的场景；大多数 CX6 RoCE 部署用 DCQCN 更实际。[C9]

6.4 TIMELY：RTT 型方案的代表

TIMELY 用 RTT 作为拥塞信号，按 RTT 变化调整速率；它证明了“延迟信号”在数据中心可行，但不是 CX6 的原生配置项，也没有对应的 NVIDIA `mlxconfig` 参数。[C20]

ZTR-RTT 可以看作 NVIDIA 在新平台上把 RTT 信号工程化的一种实现；两者思想相近，但实现、参数、平台完全不同。不要把 TIMELY 的公式直接套到 DCQCN。

6.5 一张表选算法

算法	信号	平台	优点	边界	建议
DCQCN	ECN + CNP	CX6 / RoCEv2	成熟、生态完整	需 ECN/CNP QoS；PFC 可能掩盖拥塞	默认选它
QCN	L2 CNM	Legacy L2/RoCEv1	L2 内闭环	不可路由；与 DCQCN 不同	仅遗留环境
ZTR-RTT	RTT + CNP	新平台/DPA CC	少依赖交换机 ECN 配置	平台限制；参数体系不同	按平台确认
PCC	自定义	BlueField-3+	最灵活	开发/验证成本高	研究/差异化
TIMELY	RTT	研究/业界	延迟信号	非 CX6 原生	理解概念

7 参数敏感性与交互

调参前先建立“方向感”：这个参数调大是让系统更保守还是更激进？它和谁耦合？如果不知道方向，实验就变成碰运气。

7.1 敏感性矩阵

参数	调大	调小	主要风险	先看
ECN Kmin	标记更晚；缓冲/时延更高；PFC 风险上升	标记更早；吞吐可能下降；降速更频繁	PFC 先触发/时延高	ECN 标记率、队列时延
ECN Kmax	标记斜坡更长；同一队列下标记更弱	标记更早变强；更激进	过激降速	Pmax、吞吐、CNP
Pmax	高队列时标记更强	高队列时标记更弱；可能降速不足	PFC/丢包	标记率、队列
MIN_TIME_BETWEEN_CNPS	CNP 更少；反应更慢	CNP 更密；控制面开销更高	过冲/振荡	CNP 速率、RP 降速
DCE_TCP_G	改变 α 平滑/敏感度；方向依赖实现	同左；默认优先	α 抖动/反应过慢	α、CNP 到达模式
RPG_GD	单次降速更温和	单次降速更狠	降速不足/过激	CNP 后速率曲线
RPG_AI_RATE / HAI_RATE	恢复更快；更易振荡	恢复更慢；吞吐恢复差	锯齿/恢复慢	恢复时间、P99
RPG_TIME_RESET / BYTE_RESET	恢复更慢	恢复更快	恢复慢/再次拥塞	恢复斜率
PFC Xoff	PFC 更晚；占用更多缓冲；headroom 风险	PFC 更早；pause 更频繁	丢包/风暴	pause、buffer、drop
Headroom / cable length	吸收更多在途；占用更多缓冲	缓冲更省；丢包风险上升	PFC 前丢包	no-buffer-discard
Buffer pool	吸收更强；时延更高	更省；丢包风险	时延/丢包	buffer max usage

7.2 四组关键耦合

耦合	为什么	怎么处理	典型现象
ECN Kmin ↔ PFC Xoff	不同缓冲：ECN 在 Egress 队列，PFC 在 Ingress PG；不能直接比大小 [C12]	目标是 ECN 先动作、PFC 兜底；用标记率/pause/丢包验证	PFC 高 + ECN 低 → 先降 Kmin，不要先抬 Xoff
CNP 速率 ↔ AI/HAI	CNP 越密、恢复越猛，越容易振荡	CNP 最小间隔、AI/HAI 一次只动一个	降速后锯齿/过冲
RPG_GD ↔ α/g	两者都改变闭环增益	不要同时调；先默认，再只动 RPG_GD	降速过激/不足
Headroom ↔ Xoff/cable/MTU	Xoff 越高，留给在途报文的 headroom 越紧张	按最坏 MTU/线缆/速率算 headroom	PFC 配了仍丢包

7.3 调优的“支配顺序”

1. **正确性**：priority、trust、PFC 域、CNP strict、ECN TC——错一项，后面都白调。
2. **缓冲/headroom**：先保证 PFC 不会在生效前丢包；否则测到的是缓冲问题，不是 CC 问题。

3. **ECN 阈值**：Kmin/Kmax/Pmax 决定“何时给信号”，优先于端点参数。
4. **CNP QoS/速率**：反馈通道是否可靠、是否过密。
5. **DCQCN 恢复参数**：RPG_GD、AI/HAI、定时器——最后才动。
6. **PFC 微调**：Xoff/Xon 是最后 5%，不要用它掩盖 ECN 问题。

> **不要先动 RPG_GD**
>
> RPG_GD 很显眼，但它只改变“单次降速幅度”。如果 ECN 没标记、CNP 没回来，调 RPG_GD 不会让闭环工作。先确认信号链，再调增益。

7.4 不要这样调

- 不要同时改 Kmin、Pmax、RPG_GD、AI/HAI；无法归因。
- 不要用“提高 Pmax”来解决“ECN 不触发”；Pmax 只影响高队列时的强度，不改变起始点。
- 不要把 Xoff 调高来“减少 PFC”；Xoff 越高，PFC 越晚，越容易在生效前丢包。
- 不要把 buffer 加到“永不丢包”；大缓冲会抬高时延、拖慢 ECN。
- 不要用平均吞吐验收；P99、PFC pause、Jain 公平性同样重要。
- 不要把 ZTR-RTT/PCC 的参数套到 CX6 DCQCN；两套体系不通用。

8 调优方法论：基线、实验与统计

调优不是“改一个参数看吞吐”。它是一次受控实验：先定义目标，再建立基线，再改变一个因素，最后用分布和护栏指标判断是否真的更好。

8.1 定义目标与护栏

维度	指标	目标示例	护栏
吞吐	平均/总算力利用率、goodput	达到线速的约定比例；incast 下可恢复	不能以牺牲 P99/丢包为代价
时延	P50/P99/P999、队列时延	P99 在 SLO 内	平均值会掩盖长尾
丢包	no-buffer-discard、重传、out-of-sequence	lossless priority 为 0 或接近 0	任何持续丢包都是红线
公平	Jain、每流吞吐分布	J 接近 1；无流饿死	总吞吐正常也可能不公平
稳定	恢复时间、振荡幅度	阶跃/incast 后能收敛	只看稳态会漏掉振荡
可运维	PFC storm、配置持久化、告警	无 storm；重启后自动恢复	不能靠人工记忆

8.2 基线

- 记录固件/驱动/交换机 NOS、PSID、端口速率、MTU、线缆长度、PFC/ECN/priority 配置。
- 记录默认参数：`mlxconfig query`、`mlnx_qos -i`、交换机 `show roce`/`nv show qos roce`。[C16]
- 记录业务基线：单流、4:1/8:1 incast、混合流量、故障切换。
- 基线至少 3 次运行；每次有 warmup 和稳定态；不要用一次峰值当基线。

8.3 负载矩阵

场景	拓扑/负载	验证目标	注意
单流	1 发送端 → 1 接收端	验证链路/MTU/GID/基本吞吐	不能证明 CC 工作
Incast 4:1	4 发送端 → 1 接收端	触发 ECN/CNP/PFC	最常用的 CC 验收
Incast 8:1/16:1	更多发送端	压力/公平性/恢复	观察 PFC 风暴风险
Permutation	全互通/全对全	公平性与整体吞吐	接近 AI 训练 all-to-all
Burst	短时线速突发	headroom/PFC	平均速率低也可能触发
Mixed	RoCE + TCP/存储	优先级隔离	验证 PFC 不误伤
Failure	链路/端口/交换机故障	收敛/回滚	验证冗余与恢复

8.4 实验设计

1. **一次一个因素**：Kmin、Pmax、RPG_GD、AI/HAI 分开测；如果怀疑交互，用 2×2 矩阵，不要一次改四个。

2. **设置水平**：默认值、±25%、±50%（或按参数范围取安全步长）；不要跨数量级跳。
3. **重复**：每个水平至少 3 次；取中位数，记录 IQR/P99。
4. **稳态**：warmup 后至少 5–10 分钟稳定态；incast 要覆盖同步突发周期。
5. **盲测**：如果可能，让采集/分析人员不知道当前参数，减少确认偏差。
6. **回滚**：每组实验前保存原值；出现护栏指标恶化立即回滚。

8.5 统计与判断

不要用“这次比上次高 2%”做结论。看效应量、分布和护栏：吞吐提升但 P99 翻倍，不是成功；P99 改善但丢包出现，也不是成功。

维度	统计量	采样	说明
中心趋势	中位数/均值	至少 3 次	均值受长尾影响
尾部分布	P99/P999	每次运行	AI/存储更关注长尾
离散度	IQR/标准差	每次运行	方差大说明不稳定
公平性	Jain 指数	多流	总吞吐正常也可能不公平
护栏	丢包/PFC storm/重传	每次运行	任何红线直接判失败

8.6 决策规则

- **保留**：吞吐/P99 改善达到预设效应量，且丢包/PFC/公平性没有恶化。
- **观察**：指标互有胜负；扩大样本或换负载再测。
- **回滚**：出现 lossless 丢包、PFC storm、P99 超 SLO、CNP 异常。
- **固化**：连续 3 次运行 + 重启测试通过，才写入启动配置。

调优的停止条件

出现持续 `no-buffer-discard`、PFC storm、CNP 计数异常下降、业务 SLO 明显恶化时，停止调参并回滚；不要“再试一个参数”。

9 分场景调优配方

下面的配方是“起始假设”，不是通用答案。每个场景都给出：先看什么、先调什么、避免什么、用什么验收。

9.1 AI 训练 / All-Reduce

项	内容
典型症状	同步突发、incast、P99 尖刺；PFC pause 高；ECN 标记率随步同步波动
先看	ECN 标记率、PFC pause、CNP 计数、buffer max usage、GPU/NIC 利用率
起点	NVIDIA 默认 ECN/PFC/DCQCN；CNP strict；PFC 只开 RoCE priority
优先调	Kmin（按目标队列时延）→ CNP QoS → AI/HAI（仅在恢复慢时）
避免	用 PFC 硬扛 incast；把 Kmin 设得比 PFC 还晚；同时改 AI/HAI 和 RPG_GD
验收	4:1/8:1 incast、all-reduce 周期、P99、无 no-buffer-discard、无 storm

9.2 存储 / 多 QP

项	内容
典型症状	多 QP 并发、时延敏感；PFC 偶发；重传/乱序影响尾延迟
先看	PFC pause、ECN 标记、CNP、`out_of_sequence`、P99
起点	lossless 或 lossy 按业务；ECN 默认；CNP strict；PFC 只开 RoCE priority
优先调	ECN Kmin（按 RTT/队列时延）→ PFC headroom → DCQCN 恢复
避免	把存储和普通 TCP 混在一个 lossless priority；忽略慢接收端
验收	多 QP 并发、读写混合、P99、零丢包/低重传

9.3 多租户 / Lossy

项	内容
典型症状	跨 L3、租户共享、无法保证 PFC 端到端；允许重传
先看	ECN 标记、CNP、重传、租户间公平性
起点	ECN/DCQCN；不使用 PFC；CNP strict；按租户/优先级隔离
优先调	Kmin/Kmax 让 ECN 早于丢包；CNP QoS；RPG_GD 稳定降速
避免	跨 WAN/跨租户依赖 PFC；把 PFC 当 SLA 保证
验收	多租户并发、丢包可恢复、无租户饿死、P99 可接受

9.4 混合 TCP + RoCE

项	内容
典型症状	TCP 与 RoCE 共享 buffer/priority；PFC 误伤 TCP；RoCE 被 TCP 挤占
先看	priority 映射、PFC pause、ECN 标记、两类流量吞吐
起点	RoCE 独立 lossless priority；TCP 走 lossy priority；CNP strict
优先调	优先级/TC 隔离 → ECN → PFC 域

项	内容
避免	所有 priority 都开 PFC；让 TCP 进入 lossless pool
验收	混合流量下两类业务均达标；PFC 不扩散到 TCP

9.5 长 RTT / 多级 Fabric

项	内容
典型症状	BDP 大、CNP 回来慢、阈值按短距配置导致过晚/过早
先看	RTT、BDP、队列时延、CNP 往返、PFC 域
起点	按 BDP/目标队列时延重算 Kmin；CNP strict；PFC 只到边缘
优先调	Kmin（时间单位）→ CNP 优先级 → 恢复参数
避免	直接复制 100 m/单跳的阈值；把 PFC 扩到跨 spine
验收	长 RTT incast、CNP 延迟、P99、无 PFC 风暴

9.6 短流 / RPC

项	内容
典型症状	流短、对首包/尾延迟敏感；ECN 可能来不及反应
先看	P99、PFC pause、ECN 标记、CNP 延迟
起点	ECN 早期标记；避免深缓冲；CNP strict；PFC 兜底
优先调	Kmin（较低）→ 缓冲/队列 → 恢复参数
避免	用大缓冲换吞吐；让短流排在长流后面
验收	短流 P99、混合长流、无饥饿

10 调优案例与决策树

以下案例是“观察 → 假设 → 动作 → 验证”的模板。真实环境要先看计数器，不要因为症状相似就直接套动作。

10.1 案例 1：ECN 标记高、吞吐低

项	内容
观察	ECN marked 比例 > 80%，PFC pause 不高，吞吐低于 SLO
假设	Kmin 太低/Pmax 太高，端点降速过激；或拥塞确实严重
动作	先抬高 Kmin（让标记更晚），必要时降低 Pmax；一次只动一个
验证	标记率下降、吞吐恢复、P99 不恶化、无丢包
如果无效	看队列是否持续高：可能是容量/超配问题，不是 CC 参数

10.2 案例 2：PFC pause 高、ECN 标记低

项	内容
观察	prio3_tx_pause 持续高；ECN marked 很低；可能无丢包
假设	PFC 在替 ECN 干活：Kmin 太高或 ECN 未在正确 TC
动作	降低 Kmin（让 ECN 更早标记）；确认 ECN 在 RoCE TC；不要先抬高 Xoff
验证	ECN 标记上升、PFC pause 下降、吞吐/P99 稳定
如果无效	检查 ECN 是否真的启用、CNP 是否回得来、PFC 是否误配 priority

10.3 案例 3：配了 PFC 仍然丢包

项	内容
观察	`no-buffer-discard` 增长；PFC pause 有，但仍在 Xoff 后丢包
假设	headroom 不足、cable length 错、MTU/速率不匹配、Xoff 太高
动作	按最坏情况重算 headroom；核对 cable length/MTU；不要只加 Xoff
验证	no-buffer-discard 归零；PFC pause 可控；吞吐恢复
如果无效	查线缆/光模块/FEC；查是否同 priority 混入普通流量

10.4 案例 4：ECN 标记了但 CNP 不回来

项	内容
观察	交换机 ECN marked 增长；NP/CNP 计数不增长或很低；RP 不降速
假设	NP 未 enable、CNP 被丢/被 pause、DSCP/PCP 映射错、CNP 队列不 strict
动作	查 NP enable、CNP DSCP/PCP、TC6 strict、PFC 是否误开在 CNP priority
验证	CNP 计数增长；RP 降速；ECN 标记下降
如果无效	抓包/查 ACL/查交换机 CNP 计数器；确认没有中间设备重标记

10.5 案例 5：总吞吐正常但公平性差

项	内容
观察	总吞吐达标；Jain 低；部分流 P99 很差
假设	RTT 偏置、QP 数/优先级不均、PFC 反压不公平、交换机调度
动作	先按流/租户看分布；确认是否 RTT 差异；再评估 ECN/队列/调度
验证	Jain 改善；无流饿死；总吞吐不下降
如果无效	可能需要调度/整形/租户隔离，而不是继续调 DCQCN

10.6 决策树

```
吞吐低?
├─ ECN 标记率高?
│  ├─ 是 → 队列高吗?
│  │     ├─ 是 → 拥塞严重：扩容/限流/查超配
│  │     └─ 否 → Kmin 太低/Pmax 太高：抬高 Kmin，必要时降 Pmax
│  └─ 否 → PFC pause 高?
│        ├─ 是 → PFC 在替 ECN：降低 Kmin，让 ECN 先动作
│        └─ 否 → 查 CNP/QoS/主机瓶颈
└─ 丢包?
   ├─ 是 → no-buffer-discard?
   │     ├─ 是 → headroom/cable/MTU/Xoff
   │     └─ 否 → 查链路/FEC/ACL
   └─ 否 → 查公平性/尾延迟/应用
```

10.7 停止条件

- lossless priority 出现持续 `no-buffer-discard`。
- PFC storm 计数增长，或交换机大面积 pause。
- CNP 计数异常下降，ECN 标记高但发送端不降速。
- 业务 P99/吞吐超过 SLO，且 10 分钟内无法定位。
- 重启后配置未恢复，且无法在窗口内修复。
- 出现上述任一情况：停止调参，回滚，保留现场。

11 结论与检查清单

拥塞调优的终点不是“参数看起来合理”，而是“在目标负载下可重复地满足 SLO，且没有引入新的风险”。

11.1 五条结论

1. **CX6 的 RoCEv2 主线是 DCQCN**；QCN 是 L2 遗留，ZTR-RTT/PCC 属于新平台/框架。[C1][C4][C7][C9]
2. **ECN 是信号，CNP 是反馈，PFC 是兜底**；三者不能互相替代。[C3][C4][C11]
3. **阈值要按时间理解**：Kmin/C 是队列时延；同样的字节数在不同速率下意义不同。
4. **调优顺序**：正确性 → 缓冲 → ECN → CNP → DCQCN 恢复 → PFC 微调；一次只改一个变量。
5. **验收要看分布和护栏**：P99、Jain、丢包、PFC storm，而不是平均吞吐。

11.2 上线前检查清单

- ☐ 已确认 CX6 端口是 Ethernet，RoCEv2 GID 正常。
- ☐ 已记录固件/驱动/交换机 NOS/PSID/MTU/线缆长度。
- ☐ RoCE、CNP、普通流量使用不同 priority/TC；CNP strict。
- ☐ ECN 在 RoCE TC 启用；Kmin/Kmax/Pmax 单位与平台一致。
- ☐ PFC 只开 RoCE priority；两端一致；watchdog 已评估。
- ☐ headroom/cable length/Xoff/Xon 按最坏情况核对。
- ☐ DCQCN 参数使用默认基线；没有同时改多个参数。
- ☐ 已做单流、4:1/8:1 incast、混合流量、故障/重启测试。
- ☐ 已记录 P99、Jain、ECN 标记、PFC pause、no-buffer-discard。
- ☐ 已准备回滚；没有用 mlxconfig reset 作为回滚手段。

11.3 下一步

- 用附录 B 的模板做一次完整基线，不要直接上生产调参。
- 把第 8 章的实验矩阵落成可重复的脚本/流水线。
- 把第 10 章的决策树接入值班手册。
- 固件/驱动/交换机升级后，重跑基线和关键场景。

A 附录 A 公式与量纲

以下公式用于量级判断和实验设计；标注“工程公式”的条目不是 NVIDIA 官方算法定义。

A.1 A.1 常用公式

```
BDP（字节）      = C_Bps * RTT_s
队列时延（秒）    = q_bytes / C_Bps
Kmin 对应时延（秒） = Kmin_bytes / C_Bps
ECN 标记概率     = 0, q<Kmin
                   Pmax*(q-Kmin)/(Kmax-Kmin), Kmin<=q<Kmax
                   Pmax, q>=Kmax
Jain 公平性      = (sum(x_i))^2 / (n * sum(x_i^2))
DCQCN α 更新     = (1-g)*α + g   (有 CNP)
                   (1-g)*α       (无 CNP)
DCQCN 降速       = R*(1-α/2^RPG_GD)
DCQCN 恢复       = (R+Target)/2; Target += AI/HAI
```

A.2 A.2 单位

符号	单位	说明
C_Bps	字节/秒	100GbE=12.5e9；200GbE=25e9
RTT_s	秒	路径往返时间
q_bytes	字节	Egress 队列占用
Kmin/Kmax	字节（或平台单位）	Onyx absolute 用 KB；Cumulus 用 bytes
α	0–1	DCQCN 拥塞评分
RPG_GD	指数	默认 11；不是线性百分比
Jain	0–1	1=完全公平

A.3 A.3 量级示例

100GbE、RTT=100 µs：BDP=12.5e9×100e-6=1.25 MB；150 KB 的 Kmin 约对应 12 µs 队列时延。200GbE 同样 150 KB 只对应约 6 µs，所以不能直接复用 100G 的阈值。

公式的边界

BDP/队列时延公式忽略内部 speedup、cell 开销、调度和 PFC 交互；用于判断量级，不用于精确配置。精确值以交换机厂商计算和实测为准。

B 附录 B 实验记录模板

把每次调优当成一次实验记录；没有记录的“经验”，无法复用，也无法回滚。

B.1 B.1 单次实验记录

字段	内容
日期/人员	谁在什么时间做的
环境	网卡/PSID/固件/驱动/交换机 NOS/端口速率/MTU/线缆
配置	priority、PFC、ECN TC、CNP DSCP/PCP、buffer、DCQCN 参数
负载	单流/incast 比例/流数/消息大小/持续时间
变量	只写本次改变的一个因素；原值/新值
指标	吞吐、P50/P99/P999、Jain、ECN 标记、PFC pause、丢包、重传
结果	与基线比较；是否达到效应量；护栏是否恶化
结论	保留/观察/回滚；下一步

B.2 B.2 测试矩阵

场景	负载	目的	重复
单流	1→1	基线/链路	3 次
Incast 4:1	4→1	ECN/CNP/PFC	3 次
Incast 8:1	8→1	压力/公平	3 次
Permutation	全互通	公平/整体	3 次
Burst	短时线速	headroom	3 次
Mixed	RoCE+TCP	隔离	3 次
Reboot	重启	持久化	1 次

B.3 B.3 验收记录

- 记录“通过/有条件通过/不通过”，不要只写“正常”。
- 记录未达标项、影响范围、责任人和截止时间。
- 记录回滚是否演练、耗时多少。
- 把最终配置写入版本库/配置管理，不留在个人 shell 历史里。

C 附录 C 证据摘录

下表摘录支撑关键结论的原文或原始论文表述，并给出中文释义。完整上下文见附录 D。

结论	原文/论文表述（节选）	中文释义	出处
DCQCN 三角色	"The CP is typically the network switch... The NP is the receiving NIC... The RP is the sending NIC."	CP=交换机，NP=接收网卡，RP=发送网卡。	[C4]
ECN 标记曲线	"As the queue size grows past Kmin, the probability of marking increases linearly until it reaches Pmax at Kmax."	队列超过 Kmin 后标记概率线性上升，到 Kmax 达到 Pmax。	[C4]
CNP 生成	"When an ECN-marked packet arrives at the NP, a CNP is generated and sent back to the sender."	NP 看到 CE 后生成 CNP 回送源端。	[C4]
α/降速/恢复	"alpha" / "rate is cut" / "Fast Recovery / Additive Increase / Hyper Increase"	官方文档分别描述 α 更新、乘法降速和三段恢复；这是关键词摘录，不是连续引文。	[C4]
CNP 默认 QoS	"CNP_DSCP ... 48" / "CNP_802P_PRIO ... 6"	CNP 默认 DSCP 48、PCP 6。	[C5]
ZTR-RTT 信号	"RTT" + "CNP" / "Additive Increase" + "Multiplicative Decrease"	ZTR-RTT 用 RTT+CNP，AIMD；关键词摘录。	[C7]
ZTR-RTT 平台	"within the DOCA Programmable Congestion Control (PCC) framework"	ZTR-RTT 属于新平台/框架，不应假定 CX6 支持。	[C7]
PCC 平台	"DOCA PCC ... BlueField-3 platform hardware acceleration"	PCC 依赖 BlueField-3/DPA。	[C9]
Onyx ECN 单位	"minimum-absolute ... (in KBs)"	Onyx absolute ECN 阈值单位是 KB。	[C13]
Cumulus ECN 默认	"minimum buffer threshold of 150000 bytes ... maximum 1500000 bytes"	Cumulus 默认 150 KB/1.5 MB（字节表示）。	[C14]
PFC watchdog	"PFC watchdog detects abnormal back pressure ... disables PFC temporarily"	watchdog 检测异常反压并临时止损。	[C14]
ECN 协议语义	"ECN uses two bits in the IP header" / "ECT/CE"	ECN 用 IP 头 2 bit，ECT/CE。	[C19]
DCQCN 论文	"Congestion Control for Large-Scale RDMA Deployments"	DCQCN 的算法来源与设计背景。	[C18]

D 附录 D 来源与链接

以下 21 份来源是本文的事实/算法依据，访问日期 2026-10-25。NVIDIA 文档可能随版本更新；链接失效时用标题检索。

ID	文档/论文（版本）	用途	链接
C1	NVIDIA ConnectX-6 Ethernet Adapter Cards User Manual — Introduction 2022-08-31	CX6 的 PFC/ECN/QCN/RoCE 能力边界	官方页面
C2	NVIDIA MLNX_OFED — Explicit Congestion Notification (ECN) 24.10-2.1.8 LTS	ECN RP/NP sysfs 启用与参数路径	官方页面
C3	NVIDIA MLNX_OFED — Flow Control 24.10-2.1.8 LTS	PFC 配置、计数器、stall prevention	官方页面
C4	NVIDIA DOCA — DCQCN CC Algorithm 3.5.0	CP/NP/RP、ECN 标记、CNP、α、降速与恢复	官方页面
C5	NVIDIA DOCA — DCQCN CC Parameters 3.5.0	DCQCN 默认值、单位、CNP DSCP/PCP	官方页面
C6	NVIDIA DOCA — Congestion Control Infrastructure 3.5.0	DCQCN 与 ZTR-RTT 的定位、原生 CC 与可编程 CC 边界	官方页面
C7	NVIDIA DOCA — ZTR-RTT CC Algorithm 3.5.0	ZTR-RTT 的 RTT+CNP、AIMD、时间戳与 HOL 说明	官方页面
C8	NVIDIA DOCA — ZTR-RTT Parameter and Counter Configuration 3.5.0	ZTR-RTT 参数（ALPHA/MAX_DEC/MAX_INC/AI）与调优范围	官方页面
C9	NVIDIA DOCA — DOCA PCC 3.5.0	可编程 CC 的平台要求、RP/NP、DPACC、BlueField-3 限制	官方页面
C10	NVIDIA Onyx — RDMA over Converged Ethernet (RoCE) 3.10.4800	Onyx RoCE 模式、默认优先级、ECN/PFC/CNP 配置	官方页面
C11	NVIDIA Onyx — Priority Flow Control (PFC) 3.10.4800	PFC 语义、Xoff/Xon、watchdog	官方页面
C12	NVIDIA Onyx — Shared Buffers 3.10.4800	PG/TC、headroom、reserved/shared、lossless pool	官方页面
C13	NVIDIA Onyx — QoS Commands（含 RED/ECN） 3.10.4800	ECN absolute threshold 的 KB 单位、RED/ECN 命令、trust/DSCP 映射	官方页面
C14	NVIDIA Cumulus Linux 5.15 — Quality of Service 5.15	PFC 缓冲/watchdog、ECN 默认阈值、cable length	官方页面
C15	NVIDIA Cumulus Linux 5.15 — RDMA over Converged Ethernet (RoCE) 5.15	RoCE 默认配置、PFC+ECN、DSCP 26/48	官方页面
C16	NVIDIA MFT — mlxconfig 4.36.0	固件配置持久化与重启生效	官方页面
C17	NVIDIA Network Operator — Configuration Details v26.10.0	RoCE 优化默认、运行时配置非持久	官方页面
C18	Zhu et al., Congestion Control for Large-Scale RDMA Deployments ACM SIGCOMM 2015, DOI 10.1145/2829988.2787484	DCQCN 的算法来源、设计目标与背景	官方页面
C19	RFC 3168 — The Addition of Explicit Congestion Notification (ECN) to IP IETF, 2001	ECN 的协议定义、ECT/CE 语义	官方页面
C20	Mittal et al., TIMELY: RTT-based Congestion Control for the Datacenter ACM SIGCOMM 2015, DOI 10.1145/2829988.2787510	延迟型 CC 的代表与 RTT 信号	官方页面

ID	文档/论文（版本）	用途	链接
C21	RFC 5681 — TCP Congestion Control IETF, 2009	丢包型 CC 的基线（AIMD/慢启动）	官方页面

修订记录

版本	日期	说明
v1.0	2026-10-25	初版：算法地图、ECN/DCQCN 深度讲解、参数敏感性、实验方法、分场景配方、案例与决策树。

免责声明

本文基于公开 NVIDIA 文档、RFC 与 ACM 论文整理，未在读者的具体硬件/固件/业务上做实测；调优建议不构成性能承诺。生产变更前必须在同等环境验证，并准备回滚方案。